

Reprezentacje kodu źródłowego na potrzeby modeli i algorytmów głębokiego uczenia

1st inż. Jędrzej Dubiak

Wydział Elektryczny

Politechnika Warszawska

Warszawa, Polska

01169162@pw.edu.pl

2nd inż. Rafał Pytko

Wydział Elektryczny

Politechnika Warszawska

Warszawa, Polska

01178878@pw.edu.pl

3rd inż. Michał Jagodziński

Wydział Elektryczny

Politechnika Warszawska

Warszawa, Polska

01178844@pw.edu.pl

4th inż. Bartosz Dybowski

Wydział Elektryczny

Politechnika Warszawska

Warszawa, Polska

01178878@pw.edu.pl

Streszczenie—TODO na koniec

Słowa kluczowe—uczenie głębokie, reprezentacja kodu źródłowego

I. WSTĘP

Ostatnia dekada przyniosła ogromny postęp w dziedzinie uczenia maszynowego, szczególnie dzięki metodom uczenia głębokiego (Deep Learning), które zrewolucjonizowały obszary takie jak rozpoznawanie obrazów czy przetwarzanie języka naturalnego. Obecnie metody te są coraz szerzej adaptowane w inżynierii oprogramowania, gdzie wspomagają rozumienie i analizę programów komputerowych. Fundamentem sukcesu uczenia głębokiego w tym obszarze jest uczenie reprezentacji (representation learning), czyli proces przekształcania surowych danych w format, który modele mogą efektywnie przetwarzać [1].

Reprezentacja kodu źródłowego to proces transformacji tekstowej postaci kodu programu w ustrukturyzowany, ogólny format wejściowy akceptowalny dla modeli uczenia głębokiego. Modele te nie są w stanie bezpośrednio zrozumieć surowego kodu źródłowego. Dlatego kod musi zostać przekształcony w taki sposób, aby skutecznie oddawał jego właściwości strukturalne, syntaktyczne oraz semantyczne [2].

Głównym celem tworzenia reprezentacji kodu źródłowego jest umożliwienie modelom automatycznego wykrywania skomplikowanych wzorców w strukturach programów. Wybór konkretnej metody reprezentacji jest kluczowy, ponieważ determinuje on, jakie informacje o kodzie zostaną zachowane, a jakie utracone, co bezpośrednio wpływa na skuteczność algorytmu w danym zadaniu [3].

II. TAKSONOMIA I CHARAKTERYSTYKA REPREZENTACJI KODU ŹRÓDŁOWEGO

W literaturze powtarza się dobrze utrwalona klasyfikacja reprezentacji kodu źródłowego ze względu na ich strukturę oraz poziom zawartej w nich informacji. Wyróżniamy reprezentacje leksykalne, syntaktyczne, semantyczne oraz hybrydowe i inne.

Posłużmy się przykładem kodu źródłowego napisanego w języku C w celu rozróżnienia charakterystyki każdej z wymienionych reprezentacji.

```
1 void foo() {  
2     int x = source();  
3     if(x < MAX) {  
4         int y = 2*x;  
5         sink(y);  
6     }  
7 }
```

Listing 1. Przykład kodu źródłowego do analizy reprezentacji [2]

A. Reprezentacje Leksykalne (Token-based)

Reprezentacja oparta na tokenach traktuje kod źródłowy jako płaską sekwencję znaków. Reprezentacja polega na konwersji kodu na listę tokenów, gdzie pojedynczym tokenem może być pojedynczy znak lub sekwencja kilku znaków.

Można wyróżnić kilka głównych podejść do podziału kodu na tokeny [1]:

- Tokenizacja na poziomie wyrazów - token reprezentuje wyrazy lub znaki specjalne.
- Tokenizacja na poziomie znaków - token reprezentuje pojedyncze znaki.
- Tokenizacja na poziomie podśłów - kompromis między znakami, a wyrazami.
- Tokenizacja N-gram - token reprezentuje n-elementową sekwencję, np. pierwszy token to "void foo", a następny to "foo()", co dodaje informacje o kontekście sąsiadujących wyrazów.

```
1 ['void', 'foo', '(', ')', '{', 'int', 'x',  
2  '=', 'source', '(', ')', ';',  
3  'if', '(', 'x', '<', 'MAX', ')', '{', 'int',  
4  'y', '=', '2*x', ';',  
5  'sink', '(', 'y', ')', ';', '}', '}']
```

Listing 2. Reprezentacja kodu w formie ztokenizowanej na poziomie wyrazów [2]

Aby modele Deep Learning mogły uczyć się z reprezentacji leksykalnej, tokeny są przekształcane na numeryczne wektory (embeddings) za pomocą statystycznych modeli językowych, takich jak word2vec lub doc2vec. Każdy token reprezentowany jest w ten sposób jako wektor liczb rzeczywistych [2]. W przypadku analizy sekwencyjnej, sieci takie jak LSTM przetwarzają te wektory krok po kroku. Matematycznie,

stan ukryty h_i^{tok} dla i -tego tokenu obliczany jest na podstawie poprzedniego stanu ukrytego oraz wektora osadzenia bieżącego słowa $w(x_i)$ [4]:

$$h_i^{tok} = LSTM(h_{i-1}^{tok}, w(x_i)) \quad (1)$$

B. Reprezentacje Syntaktyczne (Tree-based)

Reprezentacja syntaktyczna modeluje kod poprzez uwzględnienie jego hierarchicznej struktury oraz reguł gramatyki użytego języka programowania.

Najpopularniejszą strukturą w tym podejściu jest drzewo AST (Abstract Syntax Tree). Zamiast przetwarzać kod jako płaski ciąg znaków, AST konwertuje go w drzewo, w którym węzły reprezentują konkretne konstrukcje języka, takie jak wyrażenia, instrukcje, deklaracje [1]. Rysunek 1 przedstawia przykładową reprezentację kodu źródłowego w formie drzewa AST.

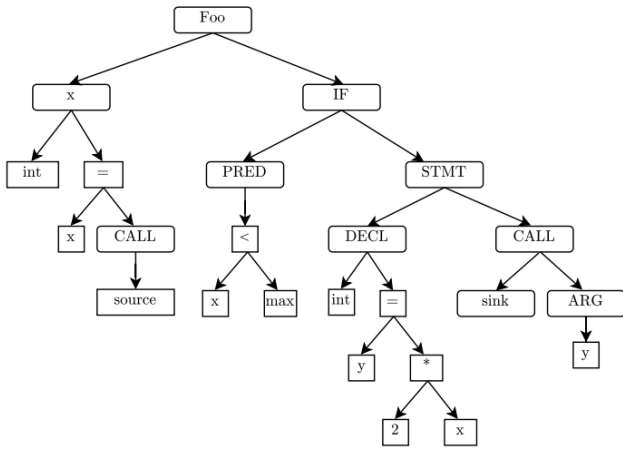


Fig. 1. Reprezentacja kodu bazująca na strukturze drzewiastej [2]

Drzewiasta struktura tego sposobu reprezentacji kodu źródłowego wymaga zastosowania odpowiednich modeli potrafiących obsłużyć dane hierarchiczne. Modele te – do których należą np. sieci Tree-LSTM, rekurencyjne sieci neuronowe RvNN czy oparte na drzewach sieci konwolucyjne TBCNN – obliczają osadzenia (embeddings) węzłów najczęściej w sposób rekurencyjny "od dołu do góry" (bottom-up), czyli propagując informacje od liści w stronę korzenia [3]. Formalizując to podejście na przykładzie binarnego drzewa AST w modelu Tree-LSTM, węzeł i agreguje informacje równolegle ze swojego lewego (L) i prawego (R) dziecka [4]:

$$(h_i^{ast}, c_i^{ast}) = LSTM([h_{iL}^{ast}; h_{iR}^{ast}], [c_{iL}^{ast}; c_{iR}^{ast}], w(x_i)) \quad (2)$$

Gdzie operator $[:]$ oznacza konkatencję wektorów stanów ukrytych (h) oraz komórek pamięci (c) dzieci, a $w(x_i)$ to wektor osadzenia węzła-rodzica.

C. Reprezentacje Semantyczne (Graph-based)

Reprezentacja grafowa modeluje kod źródłowy z naciskiem na jego semantykę oraz relacje między instrukcjami. W tym

podejściu program transformowany jest w graf, w którym węzły najczęściej reprezentują bloki kodu lub konkretne instrukcje, a krawędzie określają przepływ sterowania lub danych.

Istnieje wiele wariantów reprezentacji grafowej, można wyróżnić reprezentacje oparte na przepływie sterowania (Control Flow Graph, CFG), przepływie danych (Data Flow Graph, DFG) czy grafie zależności programowej (Program Dependency Graph, PDG), łączącym cechy obydwu podejść [4]. Rysunek 2 przedstawia przykładową reprezentację kodu źródłowego w formie grafu przepływu sterowania CFG oraz grafu zależności programowej PDG.

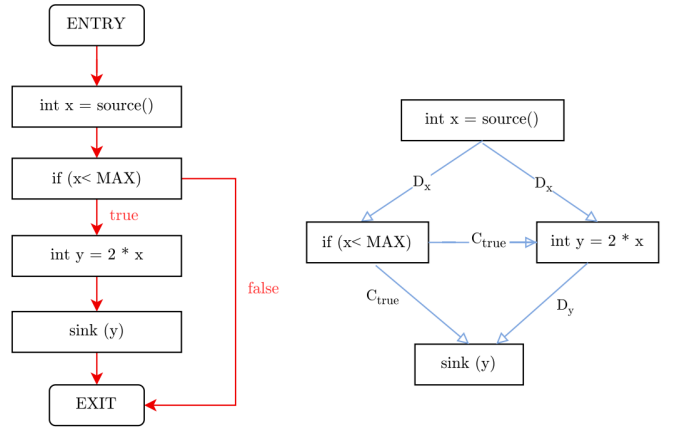


Fig. 2. Reprezentacja kodu bazująca na grafach przepływu CFG i PDG [2]

Aby skutecznie uczyć się z tak złożonych i nieliniowych struktur, stosuje się modele takie jak Gated Graph Neural Networks (GGNN). Wykorzystują one iteracyjny mechanizm przekazywania wiadomości (ang. *message passing*). W danej rundzie t , węzeł v odbiera sumę wiadomości m od swoich sąsiadów $N(v)$ w grafie, a następnie aktualizuje swój stan ukryty za pomocą bramkowanej jednostki rekurencyjnej (np. GRU) [4]:

$$m_{v,t+1} = \sum_{v' \in N(v)} W_{l_e} h_{v',t} \quad (3)$$

$$h_{v,t+1}^{cfg} = GRU(h_{v,t}^{cfg}, m_{v,t+1}) \quad (4)$$

D. Reprezentacje Hybrydowe i Pośrednie

Podejścia hybrydowe integrują wiele modalności naraz w celu uzyskania kompleksowej reprezentacji kodu źródłowego. Takie rozwiązanie kompensuje wady pojedynczych metod, łącząc wiedzę o sekwencji, gramatyce i logice wykonania.

Przykładem tego podejścia jest sieć MMAN (Multi-Modal Attention Network) zaproponowana przez Wana i in. [4]. Przetwarza ona każdą reprezentację dedykowanym jej modelem (LSTM dla tokenów, Tree-LSTM dla AST oraz GGNN dla CFG), a następnie wykorzystuje mechanizm uwagi (ang. *attention*). Mechanizm ten przypisuje wagi (α) poszczególnym elementom wewnątrz każdej modalności, co pozwala zidentyfikować fragmenty najbardziej istotne dla semantyki programu.

Ostateczna, wielomodalna reprezentacja programu x powstaje poprzez konkatencję średnich ważonych z każdego modelu i transformację przez macierz wag W [4]:

$$x = W[\sum_i \alpha_i^{tok} h_i^{tok}, \sum_i \alpha_i^{ast} h_i^{ast}, \sum_i \alpha_i^{cfg} h_i^{cfg}] \quad (5)$$

Dzięki takiej fuzji model jest w stanie jednocześnie skupić uwagę na istotnej nazwie zmiennej z reprezentacji leksykalnej, kluczowym węźle operacyjnym z drzewa AST oraz strategicznym rozgałęzieniu logiki w grafie CFG.

III. ZASTOSOWANIE REPREZENTACJI KODU ŹRÓDŁOWEGO W PROBLEMACH INŻYNIERII OPROGRAMOWANIA

tutaj o taskach SE w jakich stosuje się reprezentację kodu źródłowego czyli wykrywanie podatności, podsumowywanie kodu, detekcja klonowania itp.

IV. ANALIZA SKUTECZNOŚCI REPREZENTACJI KODU ŹRÓDŁOWEGO W PROBLEMACH INŻYNIERII OPROGRAMOWANIA

W celu weryfikacji wcześniej omawianych metod reprezentacji kodu przeprowadzono badanie empiryczne. Problemem badawczym była klasyfikacja kodu źródłowego, polegająca na przypisaniu danego programu do jednej z klas odpowiadającej problemowi algorytmicznemu.

A. Zbiór danych i przetwarzanie wstępne

Do wykonania eksperymentów wykorzystano zbiór danych Project CodeNet, który zawiera kilkanaście milionów próbek kodu z platform konkursowych i został stworzony z myślą o trenowaniu modeli w zadaniach analizy oprogramowania. Z tej bazy wybrano wyłącznie kody źródłowe napisane w języku Python, będące poprawnym rozwiązaniem danego problemu algorytmicznego (posiadające status „Accepted”).

W ramach eksperymentu wybrano 300 najpopularniejszych klas problemów. Zbiór wyfiltrowano, używając jedynie 300 próbek na klasę, co dało łącznie bazę 90 000 programów. Dane zostały także podzielone na zbiór treningowy (80%) i testowy (20%). Przygotowano także zredukowany wariant danych liczący po 100 próbek na klasę (łącznie 30 000 programów), aby zbadać wpływ wielkości zbioru na zdolność do generalizacji modeli.

W zależności od badanej reprezentacji, zastosowano inne sposoby wstępnego przetwarzania danych:

- **Reprezentacja oparta na cechach (Feature-based):** Kod potraktowano jako tekst i poddano wektoryzacji metodą TF-IDF. Zbudowano słownik oceniając wagę tokenu proporcjonalnie do częstości jego występowania w danym programie, a odwrotnie proporcjonalnie do jego częstotliwości występowania w całym zbiorze programów. W celu redukcji wymiarowości i czasu przeprowadzenia eksperymentu ograniczono słownik do 5000 najczęstszych tokenów.
- **Reprezentacja leksykalna (Sequence-based):** Kod źródłowy ztokenizowano z wykorzystaniem podziału na

podśłowa, a następnie zamieniono na sekwencje numeryczne, przypisując każdemu tokenowi unikalny identyfikator ze słownika ograniczonego do 10 000 najczęstszych słów. Sekwencje wyrównano do stałej długości 150 tokenów.

- **Reprezentacja syntaktyczna (Tree-based):** Programy przeparsowano do postaci drzew składni abstrakcyjnej (AST). Węzłom drzewa przypisano wyłącznie informacje o ich typie strukturalnym, pozbawiając je nazw zmiennych i stałych.
- **Reprezentacja hybrydowa:** Zbudowane drzewo AST wzbogacono o informacje leksykalne (m.in. nazwy zmiennych, funkcji i literały). Do każdego węzła, oprócz wektora jego typu, dołączono wektor oryginalnego tekstu przepuszczonego przez sieć BiLSTM. Takie podejście wymusza na modelu jednoczesną analizę topologii drzewa oraz leksyki programu.

B. Przebieg eksperymentów i konfiguracja modeli

Eksperyment podzielono na cztery warianty badawcze, trenując następujące modele:

- 1) **XGBoost (baseline)** na wektorach TF-IDF jako architekturę bazową.
- 2) **BiLSTM (token-based)** dla reprezentacji leksykalnej.
- 3) **Tree-LSTM (tree-based)** dla reprezentacji syntaktycznej AST.
- 4) **Hybrydowe Tree-LSTM (hybrid)** dla połączonej reprezentacji syntaktyczno-leksykalnej.

Modele głębokie trenowano przy użyciu optymalizatora Adam, stosując mechanizm wczesnego zatrzymywania z tolerancją 5 epok i maskymalną liczbą epok równą 50.

C. Wyniki i analiza

Wyniki eksperymentów dla pełnego zbioru danych (90 000 próbek) oraz zbioru zredukowanego (30 000 próbek) przedstawiono w Tabeli I.

TABLE I
DOKŁADNOŚĆ KLASYFIKACJI I CZAS UCZENIA DLA RÓŻNYCH REPREZENTACJI KODU

Model (Reprezentacja)	Dokładność (90k próbek)	Czas uczenia (90k próbek)	Dokładność (30k próbek)
XGBoost (feature-based)	30.20%	~ 0.2 min	17.05%
BiLSTM (sequence-based)	93.39%	~ 8.5 min	93.16%
Tree-LSTM (tree-based)	93.30%	~ 133 min	90.10%
Hybrydowe Tree-LSTM (hybrid)	97.37%	~ 64 min	95.09%

Na podstawie uzyskanych wyników sformułowano poniższe wnioski:

- **Niska skuteczność wektoryzacji TF-IDF:** Model XGBoost uzyskał zaledwie 30.20% dokładności (spadek do 17.05% na zredukowanym zbiorze). Reprezentacje

oparte wyłącznie na cechach ignorują strukturę i przepływ sterowania programu, co drastycznie ogranicza ich zdolność uogólniania.

- **Wpływ wielkości zbioru na generalizację:** Modele leksykalne (BiLSTM) wykazują wyższą stabilność na małych zbiorach danych (spadek zaledwie o 0.23 punktu procentowego) w porównaniu do czystych drzew AST (spadek o 3.20 p.p.). Wynika to z faktu, że tokeny dostarczają silnego kontekstu bezpośrednio z nazw zmiennych, podczas gdy abstrakcyjna topologia drzewa wymaga więcej danych do wyuczenia ukrytych wzorców.
- **Dominacja reprezentacji hybrydowych:** Wzbogacenie struktury AST o osadzenia tekstowe pozwala na uzyskanie najwyższej dokładności (97.37% na pełnym zbiorze oraz 95.09% na zredukowanym). Wynik ten potwierdza [3], że połączenie szkieletu syntaktycznego i leksyki tworzy najbardziej kompletną reprezentację kodu.
- **Złożoność obliczeniowa a zbieżność:** Mimo że najszybszym modelem uczenia głębokiego pozostaje BiLSTM (~8.5 min), podejście hybrydowe stanowi optymalny kompromis. Zastosowanie dodatkowego kontekstu leksykalnego w grafie nie tylko maksymalizuje skuteczność, ale też o ponad połowę skraca czas treningu względem czystego Tree-LSTM (z ~133 do ~64 minut), pozwalając na znacznie szybszą zbieżność sieci.

V. PODSUMOWANIE

Wnioski dotyczące wyboru optymalnej reprezentacji w zależności od dostępnych zasobów i specyfiki problemu.

BIBLIOGRAFIA

- [1] Shanmugasundaram, D., Arivukkarasu, P., Chen, H., & Cai, H. (2025). Deep Learning Representations of Programs: A Systematic Literature Review. *ACM Computing Surveys*, 58(5), 1-37.
- [2] Samoaa, H. P., Bayram, F., Salza, P., & Leitner, P. (2022). A systematic mapping study of source code representation for deep learning in software engineering. *IET Software*, 16(4), 351-385.
- [3] Siow, J. K., Liu, S., Xie, X., Meng, G., & Liu, Y. (2022, March). Learning program semantics with code representations: An empirical study. In *2022 IEEE international conference on software analysis, evolution and reengineering (SANER)* (pp. 554-565). IEEE.
- [4] Wan, Y., Shu, J., Sui, Y., Xu, G., Zhao, Z., Wu, J., & Yu, P. (2019, November). Multi-modal attention network learning for semantic source code retrieval. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)* (pp. 13-25). IEEE.
- [5] Brauckmann, A., Goens, A., Ertel, S., & Castrillon, J. (2020, February). Compiler-based graph representations for deep learning models of code. In *Proceedings of the 29th International Conference on Compiler Construction* (pp. 201-211).
- [6] Casey, B., Santos, J. C., & Perry, G. (2025). A survey of source code representations for machine learning-based cybersecurity tasks. *ACM Computing Surveys*, 57(8), 1-41.